CodeDiff: A Fast, Robust, Production-Ready, Syntax-Aware Code Diffing Tool

Marko Ivanković
Universität Passau
Passau, Germany
marko@ivankovic.me

ABSTRACT

Code diff is the process of comparing two pieces of code. A syntax-aware diff maps one abstract syntax tree (AST) onto another. AST diffing algorithms assume the following: that the correct mapping can be expressed with the four standard operations and is the cheapest one, that computing it is affordable, and that the mapping's rendering is unique. We test each assumption on real code, to measure the viability of AST diffing.

We built a corpus of 775 real-world changes in 24 languages, each carrying a human-authored AST mapping and 317 of them a human-painted rendering. On 11.5% of changes the correct correspondence maps several nodes to several others, which insert, delete, update and move cannot express; on at least 3.1% the human-preferred mapping is strictly costlier than one an algorithm found; and 31.5% of painted changes admit at least two equally correct renderings of one settled mapping. 31.8% of AST nodes carry no text of their own and never reach the screen, so a node-level metric gives a third of its weight where no reader can see. A single whole-tree tree-edit-distance computation completes within one second for only 54.5% of 2,922 stratified-sampled code file pairs. Against this ground truth, ten configurations of seven established tools show line-level mismatch rates from 0.986% to 2.570%, with no AST-aware tool beating plain line-based `diff`.

Finally, as a tool contribution, we present CodeDiff, which combines APTED's exact tree edit distance—scoped to bounded subtree pairs rather than whole files—with GumTree's move recovery and Myers' sequence diff into one five-phase pipeline. It maps 99.87% of AST nodes correctly, and on the same line-level basis as those ten configurations it mismatches 0.446% of lines—the lowest rate of any tool measured here.

CCS CONCEPTS

• **Software and its engineering → Software maintenance tools**; *Empirical software validation*.

Conference'26, June 03–05, 2026, Ludbreg, Croatia

ACM ISBN 978-1-4503-XXXX-X/26/06
https://doi.org/XXXXXXX.XXXXXXX

KEYWORDS

source code differencing, syntax-aware diffing, empirical software engineering, source code tree edit distance

ACM Reference Format:
Marko Ivanković. 2026. CodeDiff: A Fast, Robust, Production-Ready, Syntax-Aware Code Diffing Tool. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference'26)*. ACM, New York, NY, USA, 11 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Code diffing, the process of comparing two pieces of source code and computing the set of operations that transforms one into the other, underlies modern version control and, especially, modern code review. During a review, we read the diff to see what changed and what stayed the same, so the quality of the diff directly shapes the ease and quality of the review.

The standard diff used by almost all developers and platforms today is line-based: the file is treated as a sequence of text lines, and the tool computes a shortest edit script between the two sequences using three operations, insert, delete, and update. This approach is computationally cheap and fast even on modest hardware. However, most implementations offer no move operation at all, and the tools that do support one support it only in limited ways. A whole class of common changes—extracting code into a reusable function, wrapping a block in a condition, reordering declarations—is ultimately a relocation of existing code, and without a move operation every such change must be rendered as a full deletion plus a full insertion. Reconstructing the correspondence between the deleted and inserted halves is then left to the reviewer, by eye, which is the kind of mechanical, attention-taxing work humans do least reliably. Figure 1 shows the smallest version of the problem: wrapping three unchanged lines in a conditional, which a line-based tool must report as six lines deleted and seven inserted, and which a reader has to diff again in their head to see that only the wrapper is new. One way to compute diffs with a genuine move operation is to compare the code's abstract syntax tree (AST) instead of its raw text. An AST diff maps nodes of one tree onto nodes of the other; a move is a mapping whose nodes match but whose ancestry changed. Tree diffing is a well-studied field, and Section 2 surveys the work this paper builds on. In practice, however, tree diffing has seen little adoption, and two obstacles stand between the algorithms and a tool developers use. The first is runtime: pure exact tree-diffing algorithms

Before	After

`def save(rec):`
`  db.begin()`
`  db.put(rec)`
`  db.commit()`
	`def save(rec):`
	`  if rec.dirty:`
	`  db.begin()`
	`  db.put(rec)`
	`  db.commit()`

Figure 1: Wrapping a block in a condition. Only whitespace (indentation) changed, but a line-based diff reports all of it as deleted and reinserted. An AST diff can report one insertion and a move.

are $O(n^3)$ in the number of nodes. The second is that an AST diff is not directly usable by a developer at all. It has to be visualised on the textual representation of the source code, and for any given AST diff there are several equally correct visualisations—so a tool must choose one, and the quality of that choice determines adoption as much as the mapping underneath it does.

We examined four groups of research questions. Each group is answered in its own section, in the order listed here.

- **Research Question Group 1 (Viability):** What percentage of real-world code changes have an optimal AST node mapping that cannot be expressed using common AST diff operations (Insert, Delete, Update, Move)? What percentage of real-world code changes have solutions that humans prefer whose cost is higher than the optimal AST diff, using the commonly accepted costs of common AST diff operations?
- **Research Question Group 2 (Speed):** What percentage of real-world source-code changes can a single, whole-tree tree-edit-distance computation complete within one second? [1]
- **Research Question Group 3 (Uniqueness):** What percentage of real-world code changes have multiple optimal AST node mappings? What percentage of real-world code changes have multiple equally correct textual visualisations of the diff? What percentage of the AST influences the textual visualisation?
- **Research Question Group 4 (State of the Art):** How accurately do current state-of-the-art code diffing tools find optimal visualisations? How fast are real-world state-of-the-art tools?

This paper makes the following contributions:

- **Dataset:** A set of 775 real-world code changes. The changes cover 24 languages and were selected using 3 different selection algorithms from 7,444 open source repositories. The repositories themselves are a combination of a curated list and every repository used by the Gentoo Linux distribution. Every change was solved by hand and the optimal map between the two ASTs is available, or if the optimal map is impossible using the common AST diff operations, a best-possible solution with a comment explaining the missing part. Additionally, 317 of those solutions also have the optimal visualisations: the spans of text a developer should see marked, independently of which AST nodes carry them. Often, two visualisations are provided: the minimal one and the maximal one, covering human preferences between equally valid solutions. The dataset, the tool used to create the dataset and the entire history of the dataset is available freely for future research. The dataset is described in detail in Section 3.
- **Evaluation:** Answers to all research questions. An analysis of the real-world code change data shape, irrespective of any concrete implementation, and a quantified comparison of several state-of-the-art code diffing tools on the human annotated dataset.
- **Tool:** CodeDiff, a new state-of-the-art tool. Combining all current research with strict engineering principles and carefully implemented to maximise performance.

[1] One second is where Nielsen's response-time limits put the boundary between an interaction that keeps a user's train of thought and one that visibly interrupts it [9]. See Section 5.

2 BACKGROUND

This section lists two types of existing work: more theoretical research, and primarily tool implementations.

2.1 Algorithms

Zhang-Shasha [12] gives the classic dynamic-programming algorithm for ordered tree edit distance. Given two rooted, ordered trees, it computes the minimum-cost sequence of node insertions, deletions, and relabelings that transforms one tree into the other, together with the mapping that realises that cost. It is exact, but its keyroot decomposition does repeated work across overlapping subproblems.

APTED [11] is the state-of-the-art exact tree-edit-distance algorithm. It improves on Zhang-Shasha by choosing, for each subproblem, the cheapest of three decomposition strategies (left path, right path, or heaviest inner path), rather than a single fixed strategy. This choice provably minimises the total work across the whole computation. APTED still costs $O(n^3)$ in the worst case for general trees, the same asymptotic bound as Zhang-Shasha, but its real-world runtime is markedly lower.

GumTree [4] targets source code directly, not general trees. Instead of one global tree-edit-distance computation, it runs a top-down phase that greedily matches large isomorphic subtrees, then a bottom-up phase that matches remaining container nodes by a Dice coefficient of their already-matched descendants. GumTree also introduces a distinct edit operation, move, and a recovery pass that pairs up deleted and inserted identical subtrees the top-down and bottom-up phases missed. For performance, it also introduces InsertTree and DeleteTree operations that are equivalent to repeatedly calling Insert or Delete operations on all children in the given subtree.

Myers' O(ND) algorithm [8] solves a different, more restricted problem: the shortest edit script between two flat sequences, not two trees. It underlies most line-based diff

tools, including Unix `diff`. Its cost scales with the size of the edit script, not the size of the inputs, so it stays fast when two sequences are similar.

2.2 Tools

Unix `diff` is the golden standard. It treats each file as a sequence of lines, computes a shortest edit script between the two sequences with Myers' algorithm, and reports it as hunks of deleted and inserted whole lines. It has no move operation and no sub-line detail.

`git diff` is the same line-based model inside version control. Its `xdiff` engine offers four algorithms, selected with `--diff-algorithm`: Myers, the default; *minimal*, which spends more time to find a shortest script; *patience*, which anchors on lines unique to both sides before recursing; and *histogram*, a faster generalisation of patience. All four emit the same output form as Unix `diff`.

Neovim's diff mode (`nvim -d`) runs the same `xdiff` line pass as `git` and then a second, character-level pass that highlights what changed inside each paired line; such IDE views are what developers see most.

BDiff [7] is a text-based tool that reasons about blocks of lines rather than single lines and reports character ranges within changed lines.

difftastic [5] and **diffsitter** [3] are a newer generation of tree-sitter-based diffing tools. Both prioritise a diff a human reads comfortably over an exact node-to-node mapping: difftastic reduces the parse tree to an s-expression and aligns it with its own structural algorithm, and diffsitter applies a related tree-sitter-based alignment over a smaller, hand-picked set of compiled-in grammars.

3 EMPIRICAL DATASET

To understand the real-world environment code diffing tools are used in, we built a dataset of 7,045,754 files in 30 languages. Of these, we sampled 775 real open source code changes and examined them in depth.

To ensure good coverage, we built two sets of open-source repositories:

- The Curated list - 100 open source repositories, selected manually from several lists of most popular repositories, enhanced further by repositories we were already aware of. This biased the list towards active, widely used codebases, with good coverage in many programming languages. The Curated list was also used as a smaller list to repeatedly test the paper's own codebase, including in automated test infrastructure.
- The Full List - 7,444 repositories, created by expanding the Curated list with all repositories present in the Gentoo Linux distribution's package list. This includes a wide array of codebases of different activity, code quality, userbase and programming language mixes.

We cloned each repository to a depth of 50 commits from every main branch. While we cannot guarantee that all repositories remain accessible, the entire list of repositories, the cloning script and all analysis is available from the paper's repository and can be replicated by readers, if desired.

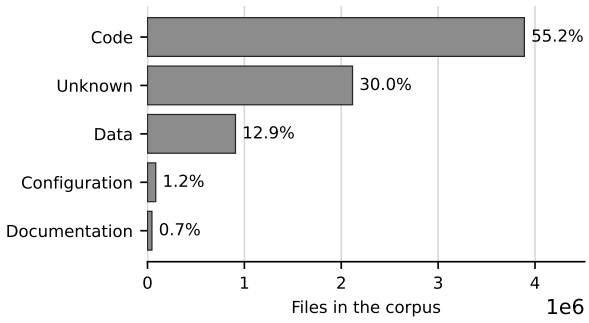

Figure 2: Distribution of file types across the corpus.

Table 1: Per-file size percentiles across all languages, code files only.

Metric	p50	p90	p99	Max
Bytes	2,353	17,703	132,227	101,352,904
LOC	61	444	2,724	196,613
AST nodes	334	3,451	23,948	905,004

We fetched, parsed and measured the corpus on a single machine: an Intel Xeon E3-1275 v5 with 64 GB of RAM, running Linux 6.8. The checkouts and the measurement database sit on 4 16 TB WDC Ultrastar DC HC550 drives in RAID 5. We record this because some results are hardware-bound, notably the wall-clock budget we hold whole-tree APTED to in Section 5.

This section describes the overall shape of the resulting dataset.

3.1 Shape of real-world files

We used a simple extension- and content-based method to categorise the files. The first thing to note about the dataset is that barely half of all files are code at all (Figure 2). The rest is configuration, data, documentation, and a substantial category of unknown type; based on limited manual inspection we believe most of those to be data. Note however that any "code" diffing tool still must support visualising changes to non-code files as well.

Restricted to code files, Table 1 reports a choice of size percentiles of bytes, lines of code, and AST nodes.

Byte size and AST node count have a strong correlation, Pearson $r = 0.8986$, which simplifies to $|\text{AST}| \approx \text{bytes}/5$ for quick estimates.

3.2 Shape of real-world file edits

Across both repository lists we analysed 114,817 commits. 31.5% of the files those commits touched were code files, giving 578,783 code-file edits, of which 75.2% were modifications; the remainder created or deleted a file and so present

Table 2: Size of real-world edits. The last row is the share of the file each edit rewrites, which is what bounds how much of a diff any tool has to reason about.

Metric	p50	p90	p99	Max
Lines changed, per file	5	63	989	2,519,229
Lines changed, per commit	11	139	2,712	4,160,787
File rewritten, per file	3.2%	33.3%	—	—

no pair of versions to map between. To stop large repositories, notably the Linux kernel, from skewing the results, each repository was analysed only 50 commits deep.

To allow for comparison with existing research, `git diff` was used to compute the number of changed lines.

Table 2 shows the sizes of real-world edits. Our results are broadly aligned with existing research. Arafat and Riehle report that 47.5% of all commits touch 10 lines or fewer [1], with the long tail stretching to tens of thousands of lines. The median edit rewrites 3.2% of the file it touches, and 59.9% of edits rewrite under 5%.

3.3 Shape of optimal file diffs

From both Curated and Full list of repositories, we fully randomly sampled 10 diffs per language. That gave 220 diffs from the Curated list across 22 languages, and 255 from the Full list across 24 languages. Every sampled diff was first manually inspected for sampling errors. At this stage, 25 diffs were rejected, mostly because programming language detection detected the wrong language. In particular TypeScript (7 rejected) and R (11) file extensions were used for other languages or data. This resulted in a dataset of 219 diffs from the Curated and 231 diffs from the Full list. As shown in Section 3.2, a pure random sample would be heavily slanted towards small changes. To account for this, the fully random sample was further augmented by a stratified sample based on the lines of code of the files being changed. The larger of the two sizes was used. The strata were: 1 to 10 lines, 11 to 30, 31 to 100, 101 to 300, 301 to 1000, 1001 to 3000 and more than 3000 lines of code. In each stratum, 10 diffs were randomly sampled per language, and inspected as above. Drawing and solving that stratified sample is still in progress; every number in this paper is reported over the 775 changes solved so far.

Each diff was then solved by hand by the authors, who have each at least 15 years of experience in software development, and have reviewed tens of thousands of real world code changes in industrial environments. Based on this experience, we did not even attempt to find a unique solution for each diff. Instead, the dataset supports multiple correct solutions for each diff, and it supports marking diffs that cannot be solved at all using the standard set of AST diff operations (Insert, Delete, Update and Move). In particular, 52 of 452 solved diffs (11.5%) require an $N{:}M$ mapping: one or more nodes from one side map to one or more nodes from the other.[2] That rate is over the 452 changes reviewed for ambiguity, not the full 775: the stratified fixtures are solved but have not yet had the ambiguity pass, and counting them as unambiguous would report a gap in the annotation as a property of the code.

Please note, diffing multiple files and detecting cross-file code moves is out of scope of this paper.

4 VIABILITY

In Section 3.3 we note that human solvers were not able to solve all diffs using the four standard AST diff operations. We used that information to answer the following research question:

RQ1.1. What percentage of real-world code changes have an optimal AST node mapping that cannot be expressed using common AST diff operations (Insert, Delete, Update, Move)?

RA1.1 (Expressibility)

11.5% of changes (52 of 452) have optimal solutions that cannot be expressed at all using the common AST diff operations (Insert, Delete, Update, Move).

Please note that this result has direct implications for other results in this paper. Because no tool can possibly solve these diffs fully, we used a charitable interpretation when evaluating tools and exclude those nodes from the evaluation. In the datasets, the nodes are left unmatched for now.

Tree diffing algorithms find the lowest cost diff, i.e. smallest number of operations needed to transform one tree into another. However, this doesn't necessarily correspond to the optimal visualisation of the diff overlayed over the text of the source code.

RQ1.2. What percentage of real-world code changes have solutions that humans prefer whose cost is higher than the optimal AST diff, using the commonly accepted costs of common AST diff operations?

The optimal cost was estimated using a state of the art diff implementation. Of the 775 fixtures with a human mapping, the two costs are equal 641 times. In 24 (3.1%) the human solution is strictly costlier. The most common reason for this is that humans treat groups of tokens as one unit, not based on their syntax or semantical meaning, but rather based on higher-level logic. A token, a common example is the `return` keyword, in a function that was completely deleted does theoretically match exactly a `return` in a newly added function. Matching the two leads to an optimal tree edit distance, but a human would find the match confusing because they typically treat the keyword as "belonging" to the logic of the function containing it. Existing tools do take notice of this as well. GumTree [4] avoids matching subtrees smaller than a threshold size to avoid such spurious mappings.

[2]The most common cause of an $N{:}M$ mapping is a refactoring, where two or more similar blocks of code are moved into a common function. The correct solution is the one that maps all of them as moved.

RA1.2 (Cost)

On at least 3.1% of the changes in the corpus (24 of 775), the human-authored mapping costs strictly more, under unit costs for insert, delete and update and a free move, than a mapping an algorithm found for the same change.

5 SPEED

The asymptotic complexity of APTED and Zhang-Shasha both is $O(n^3)$ for general trees, where n is the number of nodes in the tree. However, APTED performs considerably better in real trees because it is less sensitive to the tree shape. Given this existing direction in the literature, we wanted to further focus on source-code trees specifically and ask the following question:

RQ2. What percentage of real-world source-code changes can a single, whole-tree tree-edit-distance computation complete within one second?

One second was chosen as a budget because it is a practical limit that can actually be computed, and because it is where interaction stops feeling immediate. Nielsen's response-time limits [9] put one second at the boundary at which a user's flow of thought stays uninterrupted but the delay becomes noticeable; below about 100 ms a response reads as instantaneous. From an industry perspective, then, one second is the high end of acceptable rather than a comfortable target. Developers have also had decades to get used to instant diffs from line-based tools, so any AST diffing tool that expects to be adopted must meet a similar bar.

We sampled 2,922 real (before, after) file pairs from single-parent commits in the dataset, stratified per language and per size bucket. The buckets used in this stratification were the same as defined in Section 3.3. On each pair, we ran our highly optimised, state of the art APTED implementation, with a one second runtime budget enforced using common operating system runtime limits.

Figure 3 shows the fraction of pairs completed within one second, per size bucket. Note that the figure also provides an indication on how much the algorithm performance depends on the shape of the inputs. Scripting languages can be diffed significantly faster than normal code and diffing data files is slightly slower than code.

RA2 (Speed)

Within one second, pure tree edit distance algorithms find the diff for 97.9% of code file pairs of 11 to 30 lines. One bucket up, at 101 to 300 lines, that has already fallen to 25.3%. Over all code pairs sampled, only 54.5% complete under one second.

6 UNIQUENESS

Existing tools all provide a single diff. Line based tools mark lines as inserted, deleted or match lines 1:1. Tree based tools and algorithms have 4 basic operations: insert node, delete node, update node and match nodes 1:1. Some algorithms extend the basic operations with tree equivalents, e.g. insert tree or delete tree. No tool allows for a set of lines or nodes to match a set of lines or nodes, i.e. an N:M mapping.

Manually examining real-world inputs, we found that these operations cannot correctly express diffs that humans would identify as subjectively optimal. There are two ways in which set mappings manifest:

- Multiple equally correct 1:1 mappings within a given N:M mapping must cover one side of the mapping completely. The remaining nodes on the other side should be marked as inserted or deleted, as appropriate.
- Multiple nodes truly map to multiple nodes simultaneously.

Figure 4 is a motivating example.

The AST of the before snippet contains two `binary_expression` nodes, one a direct child of the other. The after snippet contains three. There is no objective way to determine which of the three is added and which two should be matched. The edit distance cost of all three mappings is exactly equal, and since `binary_expression` is an internal node with no rendering of its own, all three also produce the same visualisation—so human judgement cannot break the tie either.

RQ3.1. What percentage of real-world code changes have multiple optimal AST node mappings?

We count a change as admitting several optimal mappings when its ground truth records at least one *multi-map group*: a set of nodes on one side and a set on the other that the annotator judged interchangeable, so that any consistent pairing within the group is equally correct. Across the corpus the annotators recorded 218 such groups.

A second, format-independent reading points the same way. Where the annotator's mapping and the matcher's mapping for one change cost exactly the same under the unit cost model and still disagree about which nodes pair with which, the cost function cannot separate two different answers. That happens on 15 changes (1.9%).

RA3.1 (Mapping uniqueness)

In at least 11.5% of changes (52 of 452), some set of nodes admits several equally correct pairings. Independently of that annotation, on 1.9% of changes the human and algorithmic mappings tie on cost while disagreeing on the mapping, so the cost function alone cannot pick between them.

RQ3.2. What percentage of real-world code changes have multiple equally correct textual visualisations of the diff?

Settling the mapping does not settle what the reader sees. Of the 317 painted changes, 100—31.5%—carry two paintings, the annotator's finding that both are correct renderings of the same change. The other 217 carry one, and that too is a finding rather than an omission: the annotation tool records a single painting only where the painter examined the change and judged its rendering unambiguous.

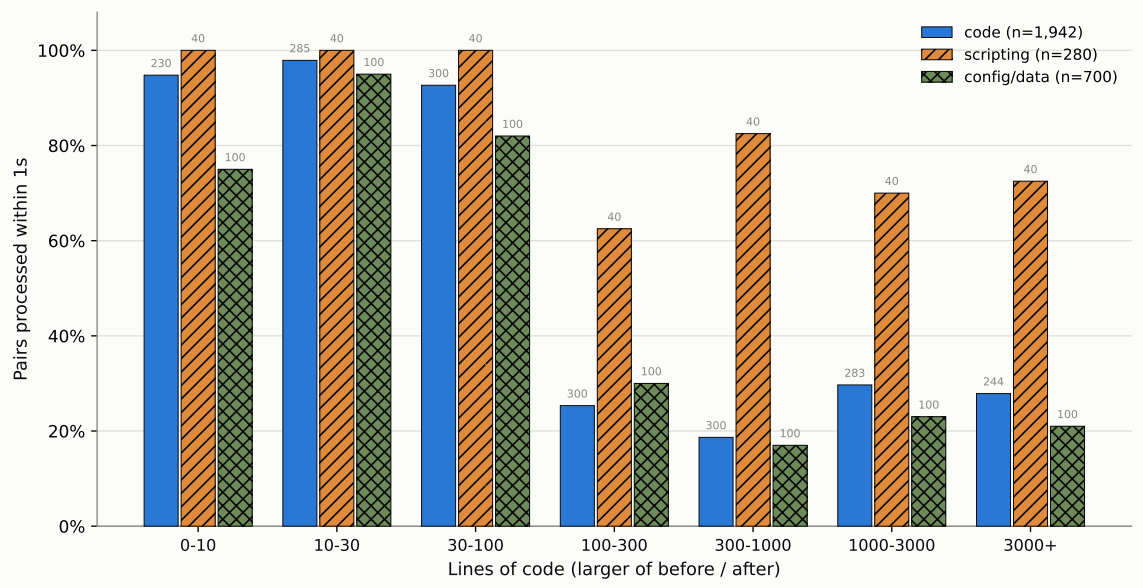

Figure 3: Fraction of sampled file pairs a whole-tree tree-edit-distance computation completes within a one-second budget, per size bucket and artifact category.

Before	After

`def is_even(x):`
`  return x > 0 \`
`  and x % 2 == 0`
	`def is_even(x):`
	`  return isinstance(x, int) \`
	`  and x > 0 \`
	`  and x % 2 == 0`

Figure 4: A change with no unique correct mapping. The before AST holds two nested `binary_expression` nodes and the after AST three; every choice of which two to match costs the same and renders the same.

RA3.2 (Visualisation uniqueness)

Of the 317 painted changes taken from real commits, 31.5% (100) admit at least two equally correct renderings.

6.1 How much of the mapping reaches the reader

RQ3.3. What percentage of the AST influences the textual visualisation?

A syntax tree contains two kinds of node. Some carry text of their own—an identifier, an operator, a literal, a comment. Others are pure structure: a `block` whose extent is exactly its children's, an `expression_statement` wrapping one expression, the chain of single-child nodes a grammar interposes between a statement and the name inside it. A rendered diff highlights regions of source text, so a node of the second kind does not appear in the output on its own. A tool that maps every displayed token correctly and every structural wrapper wrongly produces a diff a reader cannot fault; the same tool scores badly on a metric that counts all nodes equally. A node is visible if any byte inside its extent lies outside every one of its children—that is, if it contributes text of its own to the rendering.

RA3.3 (Visible share of the AST)

31.8% of the AST nodes in 775 fixtures carry no text of their own and cannot appear in a rendered diff at all: 68.2% are visible. Roughly one AST node in three is scaffolding.

The consequence is not that node-level metrics should be abandoned. It is that a rate over all nodes and a rate over visible nodes answer different questions, and any paper reporting one should say which. This paper reports both wherever it reports an accuracy figure.

7 STATE OF THE ART

RQ4.1. How accurately do current state-of-the-art code diffing tools find optimal visualisations?

RQ4.2. How fast are real-world state-of-the-art tools?

Comparison tools. We measure seven established tools, all introduced in Section 2, on the 775-fixture corpus. `git diff` is measured under each of its four algorithms, Myers, minimal, patience and histogram, giving ten configurations in all. Five configurations report at line granularity only: Unix `diff` and the four `git` algorithms. Five report sub-line detail: BDiff [7], Neovim's diff mode, and the three AST-aware tools, GumTree (v4.0.0-beta8) [4], difftastic [5] and diffsitter [3]. Our own tool's figures are reported in Section 8, on the same basis and against the same ground truth.

Line-level agreement. None of the ten configurations reports an AST-node mapping in a form the ground truth can be compared with directly, so we compare them on a common ground: which source lines each marks as touched, against the lines the human mapping touches. Coverage varies widely: of the 775 fixtures this comparison was run over, the text-based tools cover all of them, while diffsitter's compiled-in grammar set matches 504, GumTree's generator set 592, and difftastic's 752. Because those subsets differ so much, percentages are used.

We report agreement per fixture rather than pooled over lines (Table 3). Length would allow the results to be biased. *Perfect* means literally zero mismatched lines. A diff a reader can trust on sight is categorically different from one that is merely close. ≥99% and 95–99% then separate "a reader would have to look twice" from "a reader would have to check". Below 95% the tool is no longer usable.

RA4.1 (Tool accuracy)

No tool maps much more than half the corpus perfectly. By the share of fixtures with zero mismatched lines, GumTree leads at 60% and BDiff trails at 48%. The choice of line-diff algorithm barely registers: Unix `diff` and all four `git` algorithms are all within a percentage point of BDiff and `nvim -d`. Syntax-awareness alone does not guarantee that a tool recovers a change the way a human reads it: diffsitter, at 53%, is closer to the line-based tools than to GumTree.

Cost. Table 4 reports wall-clock percentiles, sorted fastest to slowest by median. Each tool is measured over the fixtures it supports rather than over one common subset, so the populations behind these rows differ—the text-based tools draw on every fixture, while an AST tool draws only on the languages its grammars cover. Two tools are reported twice: GumTree once invoked as a fresh process per fixture, the realistic cost of a single CLI invocation, and once run inside one persistent JVM across all fixtures, isolating JVM startup from GumTree's own matching cost; and BDiff likewise per process and inside one persistent Python interpreter, for the same reason.

RA4.2 (Tool cost)

The ten configurations span more than two orders of magnitude at the median, from `git`'s 2.2 ms to GumTree's 463.9 ms per invocation, and the spread widens rather than narrows into the tail: at the 99th percentile the same range runs from 11.1 ms to 5673.5 ms. Every AST-aware tool has a tail heavier than its median suggests—the ratio between a tool's own p99 and p50 is at least 20× for each of the three, against under 6× for Unix `diff` and every `git` algorithm.

We report both a cold and a warm figure for GumTree and BDiff because they answer different questions: the cold number is what a developer running a diff from the command line actually waits for, and the warm number is what the matching costs once process startup is amortised away.

Every timing number in this paper is the result of repeated runs rather than a single measurement.

8 CODEDIFF

This section is the paper's tool contribution: a syntax-aware diffing tool built against real-world measurements. It is evaluated against the same ground truth and on the same basis as Section 7's ten configurations.

The measurements in Sections 3 and 5 give CodeDiff two concrete, named design targets, stated in place of an asymptotic bound:

- **Robust:** CodeDiff must handle every file up to the measured maximum, 905,004 AST nodes, with headroom above that number.
- **Fast:** CodeDiff must compute a diff in under 400 ms for 99.99% of real-world commits.

The rest of this section reports how well CodeDiff meets the Fast target in practice, and how accurately it maps the corpus of Section 3.

CodeDiff matches two ASTs in five phases. Each phase either resolves part of the mapping directly or narrows the residual left for the next phase. Later phases only ever see nodes earlier phases left unmatched. A sixth step closes the mapping, recording the deletion or insertion implied by whatever every phase above left undecided.

Phase 1, hash-based descent. CodeDiff matches subtrees by two hash variants, in order: byte-identical subtrees, then same-shape subtrees with any leaf value. This phase is CodeDiff's own design, motivated directly by the commit-size data of Section 3: since most commits touch a small fraction of a file's lines, most of a typical file's AST is byte-identical between before and after. Resolving that identical majority with cheap hashing, before any tree-edit-distance computation runs at all, avoids paying that computation's cost on content that never changed.

Phase 2, contextual exact matching. Comments and modifiers (attributes, decorators) that immediately precede an already-matched node, and diagnostic statements (logging calls, assertions, panics) that are byte-identical on both sides, get matched directly. Neither is reachable by phase 1's structural hashing: comments and modifiers are not part of the hashed AST shape, and diagnostic statements are held back deliberately, so that matching one does not fragment a larger match around it.

Phase 3, syntax-aware subtree matching. This phase generalises three matching strategies into one engine: matching by fully-resolved name (handling overloads and duplicate names across scopes), a greedy cost-estimate matcher for containers with no name structure, such as an `if` block or a loop body, and an overlap matcher pairing import statements that share a base path. Before any of these run, large flat subtrees, such as a macro's token list or a big flat argument list, are pre-matched directly with Myers' algorithm [8], since a flat sequence has no tree structure worth exploiting. Each

Table 3: Per-fixture agreement with the human mapping, bucketed, at both granularities. The upper block is *line-level* agreement for all ten configurations. The lower block is *node-level* agreement, which only tools whose output carries sub-line structure can be scored on: a purely line-based tool has no finer signal to project onto the AST. The node metric is a per-node "did you consider this changed" projection, not mapping fidelity. "Perfect" means zero mismatches, not a rounded 100%. Each tool is scored on its own applicable subset (n), so percentages, not counts, are comparable across rows.

Tool	n	Perfect	≥99%	95–99%	<95%
Line granularity: line-only tools					
Unix `diff`	775	379 (49%)	169 (22%)	127 (16%)	100 (13%)
`git` (Myers)	775	381 (49%)	170 (22%)	127 (16%)	97 (13%)
`git` (minimal)	775	381 (49%)	170 (22%)	127 (16%)	97 (13%)
`git` (patience)	775	380 (49%)	169 (22%)	128 (17%)	98 (13%)
`git` (histogram)	775	380 (49%)	169 (22%)	129 (17%)	97 (13%)
Line granularity: tools reporting sub-line detail					
BDiff (cold, per-invocation)	775	371 (48%)	165 (21%)	139 (18%)	100 (13%)
`nvim -d`	775	391 (50%)	170 (22%)	125 (16%)	89 (11%)
diffsitter	504	266 (53%)	86 (17%)	67 (13%)	85 (17%)
difftastic	752	435 (58%)	118 (16%)	106 (14%)	93 (12%)
GumTree (cold, per-invocation)	592	356 (60%)	78 (13%)	79 (13%)	79 (13%)
Node granularity: tools reporting sub-line detail					
BDiff (cold, per-invocation)	775	478 (62%)	132 (17%)	86 (11%)	79 (10%)
`nvim -d`	775	387 (50%)	164 (21%)	111 (14%)	113 (15%)
diffsitter	504	257 (51%)	96 (19%)	82 (16%)	69 (14%)
difftastic	752	340 (45%)	147 (20%)	148 (20%)	117 (16%)
GumTree (cold, per-invocation)	592	299 (51%)	117 (20%)	100 (17%)	76 (13%)
CodeDiff	775	676 (87%)	58 (7%)	24 (3%)	17 (2%)

Table 4: Wall-clock time percentiles for the ten tool configurations, milliseconds, sorted fastest to slowest by median. CodeDiff is reported separately in Section 8.

Tool	p50	p90	p99
`git` (minimal)	2.2	4.8	10.8
`git` (Myers)	2.2	5.2	11.9
`git` (patience)	2.2	4.9	11.1
`git` (histogram)	2.2	4.9	11.2
Unix `diff`	2.5	5.6	14.2
BDiff (warm interpreter)	3.8	17.1	631.5
diffsitter	4.0	35.4	235.9
`nvim -d`	16.6	28.2	61.5
difftastic	45.3	157.1	784.3
GumTree (warm JVM)	13.6	484.7	3467.0
BDiff (cold, per-invocation)	280.8	473.8	1436.6
GumTree (cold, per-invocation)	463.9	1319.5	5673.5

accepted pair is then handed to APTED [11], scoped to that pair alone, to compute its exact internal edit script.

Phase 4, residual alignment. Whatever remains unmatched after phases 1 through 3 is resolved by a Myers-LCS alignment [8] over the residual forest, bracketed by two passes of strict bottom-up propagation: a node whose children all match the children of exactly one node on the other side is matched to it, which shrinks the residual the alignment must guess at, and is run again afterward so that pockets the alignment just resolved can still bubble up to their now-complete ancestors.

Phase 5, move-detection recovery. The final matching pass. Ordered tree edit distance cannot express a subtree that relocated across a matched boundary as anything but a delete plus an insert. This phase, adapted directly from GumTree's recovery-mappings phase [4], pairs up whole subtrees left fully deleted on one side and fully inserted on the other, when they are byte-identical and large enough to rule out coincidence.

Zhang-Shasha [12] plays no role in this pipeline as a shipped algorithm. CodeDiff's own test suite instead implements it from scratch as an independent oracle, and fuzz-tests thousands of random trees to confirm that APTED's result always matches Zhang-Shasha's, bit for bit. This checks the pipeline's most correctness-critical component against a second, independently-implemented ground truth, not only against itself.

Accuracy. Measured directly against the human mapping, CodeDiff maps 5,475,305 of 5,482,317 AST nodes correctly across the 775 fixtures, 7,012 mismatches, or 99.87% node-level accuracy. That denominator counts every AST node, including every ancestor of every change up to the root, so it partly reflects how deeply a given grammar nests. Restricted to the 68.2% of nodes RA3.3 finds visible—those carrying text of their own, and therefore able to reach the screen when the diff is rendered—the figure is 99.87%, 4,813 mismatches out of 3,740,894.

Scored on the same line-level basis as the ten configurations of Section 7, CodeDiff marks 2,917 lines differently from the human mapping over the same 775 fixtures, a rate of 0.446%, better than every tool in that table on its own applicable subset. Holding the fixture set fixed does not change that: on the 432-fixture subset every tool scored, its rate is 0.654%, against 0.741% for the best of the line-based tools and 1.713% and 2.790% for difftastic and GumTree.

Speed. Against the tools of Table 4, CodeDiff is the fastest of the three AST tools at the median, 2.0 ms against diffsitter's 4.0 ms and difftastic's 45.3 ms—and ahead of the line-based tools at that percentile too. That ordering does not hold across the distribution. diffsitter, whose narrower hand-picked grammar set does markedly less work, overtakes it at the 90th percentile (35.4 ms against 81.8 ms) and stays roughly twice as fast at the 99th (235.9 ms against 468.3 ms); difftastic is slower at every percentile below. The line-based tools are fastest from the 90th percentile upward, since they never parse or compare tree structure.

Summary. Combining established techniques rather than inventing new ones, CodeDiff reaches 99.87% AST-node accuracy against human-verified ground truth—99.87% on the nodes a reader can see—the lowest line-level mismatch rate of any tool measured on the full corpus, and a median well inside its interactive budget.

9 THREATS TO VALIDITY

Construct validity. Every rate this paper reports about ambiguity is a floor rather than an estimate. A multi-map group exists where an annotator noticed that two pairings were equally correct and recorded it; a change whose alternative mapping nobody saw is filed as unique. The same holds for the paintings. We report these as existence results and lower bounds throughout, and RA3.1's second reading—cost ties at differing mappings, which needs no annotation to detect—exists precisely because it does not depend on annotator attention. For the same reason RA1.1 and RA3.1 are scored over the 452 changes that have had an ambiguity pass rather than all 775; the stratified fixtures carry no multi-map group at all, which at the rate the reviewed lists show is an annotation gap rather than a finding about them.

RA1.2 compares the human mapping against a heuristic matcher's rather than against a proven optimum, because no whole-file exact computation is affordable at these sizes (RA2). Its rate is therefore a lower bound, and we report the 14.2% of changes on which that heuristic is itself beaten so a reader can see how loose the bound is.

Section 7 scores tools at the line level because that is the only granularity all ten configurations share. A line-level projection of a node mapping discards detail that the AST-aware tools compute and the line-based ones never had, which if anything favours the line-based tools. It also means the paintings—the only place the corpus records a second correct rendering—never enter that comparison, so RA3.2's error term sits in those tables unremoved.

Internal validity. CodeDiff is timed in-process while every external tool is timed as a whole process, so it pays no process-startup cost that the others pay in full. We state this rather than correct for it, and report GumTree and BDiff both cold and warm so that a reader can see how large the effect is for the two tools where it dominates. Per-tool mean runtimes are additionally distorted by run order, whichever tool runs first per fixture absorbing cold-cache cost; we therefore quote percentiles rather than means throughout, and every timing number is the result of repeated runs.

The ground truth is authored by the same people who built CodeDiff, which is a real risk of bias in its favour. Two things limit it: the mappings are authored against the change itself in a dedicated annotation tool with no tool output shown, and the full annotation history is published with the corpus so that any individual mapping can be inspected and disputed.

External validity. The fixture corpus is deliberately not a random sample of real commits. It concentrates hard, structurally interesting changes, because easy ones exercise nothing worth measuring—so every tail figure in this paper is a stress reading rather than an estimate of what a developer waits for on a typical commit. Where a claim needs the real distribution, we say so and mark it unvalidated, as with CodeDiff's Fast target.

The painted subset is smaller still and is not randomly drawn: painting is manual, and which changes have been painted is an artifact of the order the annotators worked in. It runs to shorter changes than the corpus as a whole, so RA3.2's rate may move in either direction as painting reaches larger changes.

Finally, the robustness evaluation covers Rust alone and a bounded node cap. The empirical-study numbers of Section 3 do cover the full 7,444-repository list, but the 775 solved changes are drawn from it by the sampling of Section 3.3 rather than being representative of it by construction.

10 RELATED WORK

Tree-diffing tools. Section 2 describes what each of the tools compared here does; what matters for positioning this work is where each one's goal differs from ours. GumTree [4] remains the reference point, and CodeDiff's move-recovery phase adapts its recovery-mappings heuristic directly: the difference is that GumTree targets research use, with broad language and backend flexibility that stays valuable well beyond what RA4.1's single corpus measures, while CodeDiff narrows to a tree-sitter-only frontend and spends the saved generality on production targets. difftastic [5] and diffsitter [3] share that parsing layer but optimise for a diff a human reads comfortably rather than for fidelity of the underlying node mapping, so RA4.1 measures them against an objective they did not choose. Against all of them, Unix `diff` and `git diff` [8] remain the baseline: on our corpus they are the fastest measured and, at the line level, more accurate than every pre-existing AST-aware tool compared. BDiff's [7] block

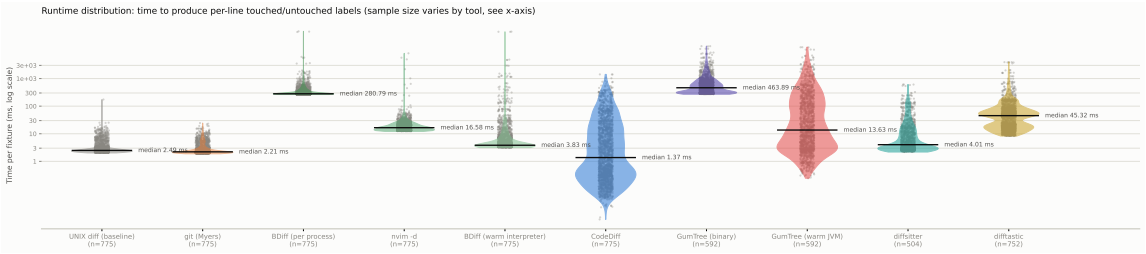

Figure 5: Full per-fixture runtime distribution (log scale), every individual repeated measurement plotted, alongside a tree-sitter-parse-only reference violin (the lower bound every AST-aware tool here must pay before its own work even starts).

awareness does not move that metric, and neither does the choice among `git`'s four algorithms [10].

Ground truth and its ambiguity. GumTree's own evaluation, and most that followed it, score a tool against a single reference mapping per change. RA1.1, RA3.1 and RA3.2 are an argument that this default understates what a corpus can be asked: on the changes where the correspondence is not one-to-one, where several mappings are equally correct, or where several renderings are, a fixed reference records a correct answer as wrong. The corpus here departs from the default in two ways—a multi-map group, which records an N:M site and admits any consistent one-to-one pairing of it, and a second, independently authored painting where the rendering is not unique—and both were added because annotating real changes forced them, not by design. We are not aware of a code-diffing ground truth that records either. RA1.2 adds a third departure from the usual assumptions: the unit cost model that ranks candidate mappings disagrees with the annotators on a measurable minority of changes.

Empirical studies. Dyer et al. [2] and Lämmel et al. [6] mine AST nodes at scale to study API usage; Arafat and Riehle [1] measure the commit-size distribution of open-source software, the distribution CodeDiff's Fast target is stated against. This paper's empirical study differs from these in purpose rather than method: it measures file- and AST-size distributions specifically to set, and then evaluate, the engineering targets of a diffing tool.

11 CONCLUSIONS

Reporting a fixed-answer accuracy metric for a syntax-aware diff is still defensible. Reporting it alone is not. That is this paper's transferable claim, and it rests on measurements that name no tool: on 11.5% of changes the correspondence is not one-to-one at all (RA1.1), on 3.1% the cheapest mapping is not the one humans judge correct (RA1.2), on the same 11.5% the mapping is not unique (RA3.1), on 31.5% of painted changes the rendering is not either (RA3.2), and 31.8% of the nodes such a metric scores never reach a reader at all (RA3.3). A field that has scored tools against one reference mapping per change has been reporting, without saying so, a number with an unquantified error term inside it. Each of these is cheap to measure on any annotated corpus, and each bounds the rate it sits beside.

Against that background, the tool contribution is deliberately modest. None of CodeDiff's matching algorithms are new: it combines APTED's exact tree edit distance, GumTree's move-recovery heuristic, and Myers' sequence diff into a single pipeline, engineered and measured against the real distribution of source code rather than an assumed worst case. On the corpus in this paper that combination reaches an accuracy, speed, and reliability comparable to or exceeding each individual predecessor it draws from, without displacing what any of them are independently good at: GumTree's language and backend flexibility, difftastic's and diffsitter's speed and readability, Unix `diff`'s universality.

The same discipline applies to the exact algorithm at the pipeline's core. CodeDiff began by running one whole-residual APTED computation per file, exactly as its asymptotic analysis invites, and removed it after measurement showed its cost tracks residual shape rather than residual size closely enough that no size threshold can gate it safely. What survives is APTED scoped to individual container pairs, where the same exactness is affordable. An exact algorithm is not automatically the right one to run at whole-file scale, and only measurement distinguishes the two regimes.

Future work includes widening the robustness corpus beyond Rust and extending the accuracy comparison to more languages GumTree also supports, but the clearest directions are the ones RA1.1, RA1.2 and RA3.2 open. The uniqueness rates are floors: the corpus establishes that ambiguity occurs at every level of the problem, but establishes it only where an annotator noticed and recorded it, so how common it actually is remains unmeasured. Detecting candidate ambiguity automatically would turn a floor into a census. RA3.2's floor rests on the 317 changes painted so far, 40.9% of the corpus, and whether the 31.5% rate rises or falls as painting reaches the larger changes is the question that would settle whether it is a floor worth much. The N:M result needs more than annotation effort. The painted format can express such a correspondence and finds 6.6% of matched regions carrying one; the mapping format records the site but only its one-to-one approximation, and no algorithm in the family measured here can emit anything else, so an output language and a

ground truth that carry the correspondence itself are both open. RA1.2 opens the last one: the comparison mapping was a heuristic's, so the rate at which humans prefer a costlier mapping is a lower bound, and a cost model that agrees with readers on those changes is unwritten.

REFERENCES

[1] Osama Arafat and Dirk Riehle. 2009. The Commit Size Distribution of Open Source Software. In *2009 42nd Hawaii International Conference on System Sciences*. 1–8. https://doi.org/10.1109/HICSS.2009.421

[2] Robert Dyer, Hridesh Rajan, Hoan Anh Nguyen, and Tien N. Nguyen. 2014. Mining billions of AST nodes to study actual and potential usage of Java language features. In *Proceedings of the 36th International Conference on Software Engineering (ICSE 2014)*. Association for Computing Machinery, New York, NY, USA, 779–790. https://doi.org/10.1145/2568225.2568295

[3] Afnan Enayet. 2020. diffsitter: A Tree-Sitter-Based AST Difftool for Meaningful Diffs. https://github.com/afnanenayet/diffsitter. Accessed 2026-07-31.

[4] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. 2014. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE '14)*. Västerås, Sweden, 313–324. https://doi.org/10.1145/2642937.2642982

[5] Wilfred Hughes. 2018. Difftastic: A Structural Diff That Understands Syntax. https://github.com/Wilfred/difftastic. Accessed 2026-07-31.

[6] Ralf Lämmel, Ekaterina Pek, and Jürgen Starek. 2011. Large-scale, AST-based API-usage analysis of open-source Java projects. In *Proceedings of the 2011 ACM Symposium on Applied Computing*. 1317–1324.

[7] Yao Lu, Wanwei Liu, Tanghaoran Zhang, Kang Yang, Yang Zhang, Wenyu Xu, Longfei Sun, Xinjun Mao, Shuzheng Gao, and Michael R. Lyu. 2025. BDiff: Block-aware and Accurate Text-based Code Differencing. arXiv preprint arXiv:2510.21094, https://github.com/BDiff/BDiff. Accessed 2026-08-23.

[8] Eugene W. Myers. 1986. An O(ND) Difference Algorithm and Its Variations. *Algorithmica* 1, 2 (1986), 251–266. https://doi.org/10.1007/BF01840446

[9] Jakob Nielsen. 1993. *Usability Engineering*. Morgan Kaufmann, San Francisco, CA.

[10] Yusuf Sulistyo Nugroho, Hideaki Hata, and Kenichi Matsumoto. 2020. How different are different diff algorithms in Git? Use `--histogram` for code changes. *Empirical Software Engineering* 25, 1 (2020), 790–823. https://doi.org/10.1007/s10664-019-09772-z

[11] Mateusz Pawlik and Nikolaus Augsten. 2015. Efficient Computation of the Tree Edit Distance. *ACM Transactions on Database Systems* 40, 1, Article 3 (2015), 3:1–3:40 pages. https://doi.org/10.1145/2699485

[12] Kaizhong Zhang and Dennis Shasha. 1989. Simple Fast Algorithms for the Editing Distance between Trees and Related Problems. *SIAM J. Comput.* 18, 6 (1989), 1245–1262. https://doi.org/10.1137/0218082